SocketCAN: Software-Only CAN/CAN FD Communication

Technical Report

Exploring Linux SocketCAN Framework with Virtual CAN Interface

Ubuntu 22.04.5 LTS Implementation

Date: August 2026

1. Introduction

This project demonstrates software-only CAN (Controller Area Network) and CAN FD (Flexible Data-rate) communication using the Linux SocketCAN framework. Instead of requiring physical CAN hardware, the implementation uses virtual CAN interfaces (vcan0) to enable multiple software nodes to communicate over a simulated CAN bus.

The project is designed for embedded systems engineers and automotive software developers who need to develop and test CAN-based applications before physical hardware is available. SocketCAN provides a standard Linux networking interface for CAN communication, making it ideal for rapid prototyping and testing.

2. SocketCAN Overview

2.1 What is SocketCAN?

SocketCAN is the Linux kernel's standard framework for CAN communication. It treats CAN controllers as network devices, similar to Ethernet (eth0) or WiFi (wlan0), allowing applications to use standard Linux socket APIs for CAN communication rather than vendor-specific drivers.

The architecture follows: CAN Application → SocketCAN Socket → Linux CAN Subsystem → CAN Interface (can0/vcan0)

2.2 Why CAN as a Network Interface?

Linux provides a unified interface for different types of hardware. By representing CAN controllers as network interfaces, SocketCAN achieves consistency, standard socket programming, hardware abstraction, and integration with Linux network utilities. This unified approach simplifies development and testing.

2.3 SocketCAN vs Vendor-Specific APIs

Vendor APIs lock developers to specific hardware. SocketCAN provides: (1) High portability—works with any CAN adapter, (2) Standard socket APIs—familiar to all Linux developers, (3) Single codebase for multiple hardware targets, (4) Integration with existing Linux tools and utilities.

2.4 Advantages of SocketCAN

• Standard Linux interface—no vendor lock-in • Multiple applications access same CAN interface simultaneously • CAN message filtering supported in the kernel • Virtual CAN (vcan) enables development without hardware • Comprehensive utilities: candump, cansend, cangen • CAN FD support for higher bandwidth applications

3. System Architecture

The implementation consists of three independent software nodes communicating over a virtual CAN interface (vcan0). Vehicle ECU generates realistic vehicle data (speed, RPM, temperature) and transmits as CAN messages. Dashboard ECU receives and displays decoded messages with optional filtering and offline detection. Logger ECU independently records all CAN traffic to CSV file.

Virtual CAN Interface (vcan0):

vcan0 is a virtual network interface simulating CAN hardware. Created using: sudo modprobe vcan; sudo ip link add dev vcan0 type vcan; sudo ip link set up vcan0. This allows complete CAN development without physical CAN controllers, transceivers, or adapters.

4. CAN Message Definitions

Three CAN messages transmit vehicle data with 500ms transmission period:

CAN ID	Signal	DLC	Unit
0x100	Vehicle Speed	2	km/h (0-120)
0x101	Engine RPM	2	rpm (800-5000)
0x102	Coolant Temperature	2	°C (20-120)

All values use little-endian (LSB first) encoding. Examples: Speed 65 km/h → [0x41, 0x00]; RPM 2450 → [0x92, 0x09]; Temperature 88°C → [0x58, 0x00]

5. Learning Challenges Results

Challenge 1: Traffic Observation

Result: PASSED. Multiple applications (Dashboard + Logger) simultaneously received CAN messages. Logger activity did not interfere with Dashboard. SocketCAN kernel efficiently delivered frames to all registered sockets.

Challenge 2: Message Filtering

Result: PASSED. Implemented CAN_RAW_FILTER socket option. Successfully filtered for Speed (0x100), RPM (0x101), and Temperature (0x102) individually. Filtering reduced kernel-to-application message delivery overhead.

Challenge 3: Unknown Message Detection

Result: PASSED. Logger recorded CAN ID 0x200 (receives all traffic). Dashboard ignored 0x200 (no display update). System remained stable with unknown CAN IDs.

Challenge 4: Transmission Rate Study

Result: PASSED. Tested intervals from 1000ms to 10ms. Higher transmission rates produced smoother dashboard updates, faster log file growth, and higher CPU usage. System remained stable at all rates.

Challenge 5: Node Failure Study

Result: PASSED. When Vehicle ECU terminated: Dashboard stopped receiving new messages; last values remained displayed; timeout mechanism detected offline status after 2 seconds; Logger stopped recording.

Challenge 6 & 7: Diagnostics and CAN FD

Result: PASSED. Implemented select() with 2-second timeout for offline detection. Successfully transmitted and received 20-byte CAN FD frames (exceeds Classical CAN's 8-byte limit). Both features demonstrated successfully.

6. CAN FD (Flexible Data-rate) Analysis

Classical CAN → max 8 bytes; CAN FD → max 64 bytes. CAN FD supports variable bit rates in data phase for faster transmission. Linux SocketCAN supports both through struct canfd_frame and CAN_RAW_FD_FRAMES socket option.

Use CAN FD for: High-bandwidth applications (video, lidar data), consolidating multiple Classical CAN messages into one, complex diagnostic data transmission, automotive applications requiring faster data rates.

Demonstrated CAN FD frame: CAN ID 0x300, Length 20 bytes, Data: 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14

7. Key Learnings and Conclusions

SocketCAN Abstraction: Treating CAN as a network interface enables standard socket programming without vendor-specific APIs.

Virtual Interfaces: vcan0 provides complete CAN development environment without physical hardware.

Multi-Application Architecture: SocketCAN efficiently delivers CAN messages to multiple simultaneous receivers.

Message Filtering: CAN_RAW_FILTER reduces kernel overhead by filtering at socket level.

Fault Detection: Timeout-based diagnostics detect node failure without heartbeat overhead.

CAN FD Capability: 64-byte payloads enable high-bandwidth applications.

Practical Implication: Embedded systems engineers can develop CAN software months before physical hardware arrives, significantly reducing development time and cost. SocketCAN represents the correct approach to CAN on Linux—standard, portable, efficient, and hardware-agnostic.